

网络协议的设计与实现（下）

北京理工大学计算机学院
金旭亮



概述

前一讲，介绍了如何设计自己的网络通讯协议，以及如何使用序列化技术，实现通讯数据对象与ByteBuf之间的相互转换。

本讲则将上一讲的成果用于实际开发，通过示例展示Netty客户端与服务端之间，如何基于自定义网络协议，实现客户端与服务端之间的相互通讯。

基本流程是这样的：

- ▶ 客户端连接上服务器之后，将用户名与密码发给服务端，服务端校验密码后，发回“登录成功”或“登录失败”的响应消息。
- ▶ 客户端收到服务器响应之后，将登录结果保存于通道的attribute中，这样，就可以接着进行后继的必须要登录之后才可以进行的业务数据交换了。

将编码与解码功能，抽取为独立的类

```
//用于完成编码与解码的工作
public class PacketCodec { 12 usages
    //固定的魔数值
    private static final int MAGIC_NUMBER = 0xABCDABCD; 1 usage

    //将Packet序列化到ByteBuf中，如果out参数为null，内部实例化一个byteBuf，
    //否则，使用传入的ByteBuf保存数据
    @ > public static ByteBuf encode(Packet packet, ByteBuf out) {...}

    //从ByteBuf中反序列化，得到一Packet对象（本质上是其
    @ > public static Packet decode(ByteBuf byteBuf) {...}
}
```

特别注意一下解码过程对可读数据不够时的处理

```
//从ByteBuf中反序列化, 得到一Packet对象
public static Packet decode(ByteBuf byteBuf) {
    //依据自定义协议, 计算指令头的总长度
    //MAGIC_NUMBER(4)+版本号 (1) +序列化算法 (1) +指令Id (1) +数据包长度 (4)
    int headerLength = 11;
    if(byteBuf.readableBytes() < headerLength){
        //读取的数据不够
        return null;
    }
    //跳过MAGIC_NUMBER
    byteBuf.skipBytes(4);
    //跳过版本号 (本例用不上)
    byteBuf.skipBytes(1);
    //获取序列化算法id
    byte serializerAlgorithm = byteBuf.readByte();
    //依据序列化算法id, 获取对应的序列化器实例
    // (比如, JSON序列化, 则使用JSONSerializer对象)
    var serializer = Serializer.getSerializer(serializerAlgorithm);
    //获取指令Id
    byte command = byteBuf.readByte();
    //依据指令Id, 获取指令的具体类型 (Class对象)
    var requestType = Command.getRequestType(command);
```

```
//获取数据包长度
int length = byteBuf.readInt();
//记录下当前读到的位置
byteBuf.markReaderIndex();
if(byteBuf.readableBytes() < length){
    //如果读取的数据不够, 则不执行后面的代码,
    //回到上次读取指令头结束时的位置
    byteBuf.resetReaderIndex();
    return null;
}
//数据已经足够了, 则读取指令包的具体数据
byte[] bytes = new byte[length];
byteBuf.readBytes(bytes);

if (requestType != null && serializer != null) {
    //反序列化, 得到具体的Packet对象
    return serializer.deserialize(requestType, bytes);
}
return null;
}
```



当可读取的数据不足时, 不要进行后续的处理, 并且应该“记下”当前已经读取数据的位置。

编写编码与解码Handler

```
//解码Netty传过来的ByteCode，将其解码为特定的Pack类型，
//比如LoginRequestPacket和LoginResponsePacket
public class PacketDecoder extends ByteToMessageDecoder {
    @Override
    protected void decode(ChannelHandlerContext ctx, ByteBuf in,
                          List<Object> out) throws Exception {
        System.out.println("PacketDecoder接收到的远方发来的byteBuf");
        System.out.println(ByteBufUtil.prettyHexDump(in));
        var packet = PacketCodec.decode(in);
        System.out.println("PacketDecoder反序列化得到的Packet:" + packet);
        out.add(packet);
    }
}
```

```
//负责将一个Packet对象序列化到ByteBuf中，然后再发送出去
public class PacketEncoder extends MessageToByteEncoder<Packet> {
    @Override
    protected void encode(ChannelHandlerContext ctx, Packet msg, ByteBuf out) {
        System.out.println("PacketEncoder收到的Packet:" + msg);
        PacketCodec.encode(msg, out);
        ctx.writeAndFlush(out);
        System.out.println("PacketEncoder最终发送出去的ByteBuf: "
                          + ByteBufUtil.prettyHexDump(out));
    }
}
```

添加“登录响应”相关类型

```
//登录响应数据包
@Data
public class LoginResponsePacket extends Packet {
    private Boolean isSuccess;
    private String message;
    @Override
    public Byte getCommand() {
        return Command.LOGIN_RESPONSE;
    }
}
```

```
//在这里，集中定义指令集，可以定义最多256个指令
public interface Command {
    //登录请求指令id
    Byte LOGIN_REQUEST = 1;
    //登录响应id
    Byte LOGIN_RESPONSE = 2;

    //依据指令id，获取具体指令实现类型
    static Class<? extends Packet> getRequestType(byte command){
        if(command == LOGIN_REQUEST){
            return LoginRequestPacket.class;
        }
        if(command == LOGIN_RESPONSE){
            return LoginResponsePacket.class;
        }
        return null;
    }
}
```

服务端代码

服务端处理登录请求的Handler

```
// 此Handler接收承载了用户登录信息的LoginRequestPacket，校验用户名与密码，然后，生成响应对象
public class LoginRequestHandler extends SimpleChannelInboundHandler<LoginRequestPacket> {
    @Override
    protected void channelRead0(ChannelHandlerContext ctx, LoginRequestPacket msg) {
        System.out.println("LoginRequestHandler收到:" + msg);
        // 生成响应对象
        var response = new LoginResponsePacket();
        if (msg.getUserName().equals("testUser") && msg.getPassword().equals("123456")) {
            response.setIsSuccess(true);
            response.setMessage("LoginRequestHandler: 登录成功! ");
        } else {
            response.setIsSuccess(false);
            response.setMessage("LoginRequestHandler: 登录失败，请检查用户名与密码是否正确。");
        }
        System.out.println("LoginRequestHandler生成响应对象: " + response);
        // 转发给下一个Handler—PacketEncoder，发送给客户端
        ctx.writeAndFlush(response);
    }
}
```


测试服务端管线

```
@Test
public void testServerPipeline() {
    ChannelInitializer<EmbeddedChannel> initializer = new ChannelInitializer<>() {
        @Override
        protected void initChannel(EmbeddedChannel ch) throws Exception {
            ch.pipeline().addLast(new PacketDecoder());
            ch.pipeline().addLast(new LoginRequestHandler());
            ch.pipeline().addFirst(new LoginResponseHandler());
            ch.pipeline().addFirst(new PacketEncoder());
        }
    };
    var channel = new EmbeddedChannel(initializer);
    //创建登录请求对象
    LoginRequestPacket loginRequestPacket = new LoginRequestPacket();
    loginRequestPacket.setUserId(1000);
    loginRequestPacket.setUserName("testUser");
    loginRequestPacket.setPassword("123456");

    //模拟客户端发来的数据，Netty是会将其转换为ByteBuf的，这里就直接实例化了。
    var clientSendData= PacketCodec.encode(loginRequestPacket,null);
    //将数据传入管线
    channel.writeInbound(clientSendData);
    //结束测试
    channel.finish();
    channel.close();
}
```

这里，先用现有的组件构建管线，然后使用EmbeddedChannel测试管线是否工作正常。

运行截图

通过仔细查询单元测试代码，可以清楚地看到客户端传来的数据，被PacketDecoder解码为LoginRequestPacket，再由LoginRequestHandler进行校验，生成LoginResponsePacket，传给PacketEncoder，发送出去。

✓ Tests passed: 1 of 1 test – 346 ms

"C:\Program Files\Java\jdk-21\bin\java.exe" ...

PacketDecoder接收到的远方发来的byteBuf

```
+-----+
| 0 1 2 3 4 5 6 7 8 9 a b c d e f |
+-----+
|00000000| ab cd ab cd 01 01 01 00 00 00 51 7b 22 63 6f 6d |.....Q{"com|
|00000010| 6d 61 6e 64 22 3a 31 2c 22 70 61 73 73 77 6f 72 |mand":1,"passwor|
|00000020| 64 22 3a 22 31 32 33 34 35 36 22 2c 22 75 73 65 |d":"123456","use|
|00000030| 72 49 64 22 3a 31 30 30 30 2c 22 75 73 65 72 4e |rId":1000,"userN|
|00000040| 61 6d 65 22 3a 22 74 65 73 74 55 73 65 72 22 2c |ame":"testUser",|
|00000050| 22 76 65 72 73 69 6f 6e 22 3a 31 7d |"version":1} |
+-----+
```

PacketDecoder反序列化得到的Packet:LoginRequestPacket(userId=1000, userName=testUser, password=123456)

LoginRequestHandler收到:LoginRequestPacket(userId=1000, userName=testUser, password=123456)

LoginRequestHandler生成响应对象: LoginResponsePacket(isSuccess=true, message=LoginRequestHandler:登录成功!)

PacketEncoder收到的Packet:LoginResponsePacket(isSuccess=true, message=LoginRequestHandler:登录成功!)

PacketEncoder最终发送出去的ByteBuf:

```
+-----+
| 0 1 2 3 4 5 6 7 8 9 a b c d e f |
+-----+
|00000000| ab cd ab cd 01 01 02 00 00 00 5a 7b 22 63 6f 6d |.....Z{"com|
|00000010| 6d 61 6e 64 22 3a 32 2c 22 69 73 53 75 63 63 65 |mand":2,"isSucce|
|00000020| 73 73 22 3a 74 72 75 65 2c 22 6d 65 73 73 61 67 |ss":true,"messag|
|00000030| 65 22 3a 22 4c 6f 67 69 6e 52 65 71 75 65 73 74 |e":"LoginRequest|
|00000040| 48 61 6e 64 6c 65 72 3a e7 99 bb e5 bd 95 e6 88 |Handler:.....|
|00000050| 90 e5 8a 9f ef bc 81 22 2c 22 76 65 72 73 69 6f |.....","versio|
|00000060| 6e 22 3a 31 7d |n":1} |
+-----+
```

Process finished with exit code 0

正式编写服务端程序

前面的测试代码，证明了管线如预期那样工作，现在，就可以编写服务端程序，让其监听9000端口.....

```
public class MyNettyServer {
    private static final int PORT = 9000;
    public static void main(String[] args) throws InterruptedException {
        NioEventLoopGroup bossGroup = new NioEventLoopGroup();
        NioEventLoopGroup workerGroup = new NioEventLoopGroup();
        ServerBootstrap serverBootstrap = new ServerBootstrap();
        serverBootstrap
            .group(bossGroup, workerGroup)
            .channel(NioServerSocketChannel.class)
            .handler(new LoggingHandler(LogLevel.INFO))
            .childHandler(new ChannelInitializer<NioSocketChannel>() {
                @Override
                protected void initChannel(NioSocketChannel ch) throws Exception {
                    ch.pipeline().addLast(new PacketDecoder());
                    ch.pipeline().addLast(new LoginRequestHandler());
                    //编码器应该放在管线开头
                    ch.pipeline().addFirst(new PacketEncoder());
                }
            });
        var bindResult = serverBootstrap.bind(PORT).addListener(future -> {
            if (future.isSuccess()) {
                System.out.println("端口[" + PORT + "]绑定成功!");
            } else {
                System.err.println("端口[" + PORT + "]绑定失败!");
            }
        });
        bindResult.channel().closeFuture().sync();
        bossGroup.shutdownGracefully();
        workerGroup.shutdownGracefully();
    }
}
```

客户端代码

客户端发送登录请求

```
//此Handler负责发送登录请求
public class MyNettyClientLoginRequestHandler extends ChannelInboundHandlerAdapter {

    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        //检查是否已经登录成功?
        var loginResult = ctx.channel().attr(MyNettyClient.clientKey).get();
        if (loginResult == null) {
            //如果没有登录,则创建登录对象
            LoginRequestPacket loginRequestPacket = new LoginRequestPacket();
            loginRequestPacket.setUserId(1000);
            //用户名为testUser,登录成功,为admin,登录会失败
            //loginRequestPacket.setUserName("testUser");
            loginRequestPacket.setUserName("admin");
            loginRequestPacket.setPassword("123456");
            var byteBuf = PacketCodec.encode(loginRequestPacket, null);
            //发送登录请求
            ctx.writeAndFlush(loginRequestPacket);
            //释放ByteBuf
            ReferenceCountUtil.release(byteBuf);
        } else {
            System.out.println("已发出过登录请求,登录结果为: " + loginResult);
        }
    }
}
```

客户端处理登录响应的Handler

```
//当服务端传回响应之后，此处理程序接收响应，  
//并依据收到的响应数据，将登录结果保存到通道的Attribute集合中  
public class LoginResponseHandler extends  
    SimpleChannelInboundHandler<LoginResponsePacket> {  
    @Override  
    protected void channelRead0(ChannelHandlerContext ctx,  
                                LoginResponsePacket msg) {  
        System.out.println("LoginResponseHandler收到数据: " + msg);  
        var info = "";  
        if (msg.getIsSuccess()) {  
            info = "success";  
        } else {  
            info = "fail";  
        }  
        ctx.channel().attr(MyNettyClient.clientKey).set(info);  
        System.out.println("LoginResponseHandler设置登录结果: " + info);  
    }  
}
```

测试客户端管线

```
@Test
public void testClientPipeline() {
    ChannelInitializer<EmbeddedChannel> initializer = new ChannelInitializer<>() {
        @Override
        protected void initChannel(EmbeddedChannel ch) throws Exception {
            ch.pipeline().addLast(new PacketDecoder());
            ch.pipeline().addLast(new LoginResponseHandler());
            ch.pipeline().addFirst(new PacketEncoder());
        }
    };
    var channel = new EmbeddedChannel(initializer);
    //生成服务端响应对象
    LoginResponsePacket loginResponsePacket = new LoginResponsePacket();
    loginResponsePacket.setIsSuccess(true);
    loginResponsePacket.setMessage("登录成功");
    //编码为ByteBuf
    var clientRecvData= PacketCodec.encode(loginResponsePacket,null);
    //模拟客户端收到了数据
    channel.writeInbound(clientRecvData);
    channel.finish();
    channel.close();
}
```


单元测试的输出

✓ Tests passed: 1 of 1 test – 289 ms

```
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
```

PacketDecoder接收到的远方发来的byteBuf

```

+-----+
| 0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f |
+-----+
|00000000| ab cd ab cd 01 01 02 00 00 00 43 7b 22 63 6f 6d | .....C{"com|
|00000010| 6d 61 6e 64 22 3a 32 2c 22 69 73 53 75 63 63 65 |mand":2,"isSucce|
|00000020| 73 73 22 3a 74 72 75 65 2c 22 6d 65 73 73 61 67 |ss":true,"messag|
|00000030| 65 22 3a 22 e7 99 bb e5 bd 95 e6 88 90 e5 8a 9f |e":".....|
|00000040| 22 2c 22 76 65 72 73 69 6f 6e 22 3a 31 7d      |","version":1} |
+-----+

```

PacketDecoder反序列化得到的Packet:LoginResponsePacket(isSuccess=true, message=登录成功)

LoginResponseHandler收到数据: LoginResponsePacket(isSuccess=true, message=登录成功)

LoginResponseHandler设置登录结果: success

```
Process finished with exit code 0
```

编写客户端功能代码

```
public class MyNettyClient {
    //用于保存登录结果
    static AttributeKey<Object> clientKey =
        AttributeKey.newInstance("loginResult");

    public static void main(String[] args) {
        NioEventLoopGroup workerGroup = new NioEventLoopGroup();
        Bootstrap bootstrap = new Bootstrap();
        bootstrap.group(workerGroup)
            .channel(NioSocketChannel.class)
            .handler(new ChannelInitializer<NioSocketChannel>() {
                @Override
                protected void initChannel(NioSocketChannel ch) throws Exception {
                    ch.pipeline().addLast(new MyNettyClientLoginRequestHandler());
                    ch.pipeline().addLast(new PacketDecoder());
                    ch.pipeline().addLast(new LoginResponseHandler());
                    ch.pipeline().addFirst(new PacketEncoder());
                }
            });
        bootstrap.connect("127.0.0.1", 9000).syncUninterruptibly()
            .addListener((ChannelFutureListener) future -> {
                if (future.isSuccess()) {
                    System.out.println("连接成功");
                } else {
                    System.out.println("连接失败");
                }
            });
    }
}
```

测试完整的网络应用程序-客户端

Run MyNettyServer x MyNettyClient x

用户名admin, 登录失败

```
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
PacketEncoder收到的Packet:LoginRequestPacket(UserId=1000, userName=admin, password=123456)
连接成功
PacketDecoder接收到的远方发来的byteBuf
+-----+
| 0 1 2 3 4 5 6 7 8 9 a b c d e f |
+-----+
|00000000| ab cd ab cd 01 01 02 00 00 00 85 7b 22 63 6f 6d |.....{"com|
|00000010| 6d 61 6e 64 22 3a 32 2c 22 69 73 53 75 63 63 65 |mand":2,"isSucce|
|00000020| 73 73 22 3a 66 61 6c 73 65 2c 22 6d 65 73 73 61 |ss":false,"messa|
|00000030| 67 65 22 3a 22 4c 6f 67 69 6e 52 65 71 75 65 73 |ge":"LoginReques|
|00000040| 74 48 61 6e 64 6c 65 72 3a e7 99 bb e5 bd 95 e5 |tHandler:.....|
|00000050| a4 b1 e8 b4 a5 ef bc 8c e8 af b7 e6 a3 80 e6 9f |.....|
|00000060| a5 e7 94 a8 e6 88 b7 e5 90 8d e4 b8 8e e5 af 86 |.....|
|00000070| e7 a0 81 e6 98 af e5 90 a6 e6 ad a3 e7 a1 ae e3 |.....|
|00000080| 80 82 22 2c 22 76 65 72 73 69 6f 6e 22 3a 31 7d |..","version":1}|
+-----+
PacketDecoder反序列化得到的Packet:LoginResponsePacket(isSuccess=false, message=LoginRequestHandler:登录失败, 请检查用户名与密码是否正确。)
LoginResponseHandler收到数据: LoginResponsePacket(isSuccess=false, message=LoginRequestHandler:登录失败, 请检查用户名与密码是否正确。)
LoginResponseHandler设置登录结果: fail
```

测试完整的网络应用程序-服务端

Run

MyNettyServer x MyNettyClient x

↺

↻

📷

📄

🗑

端口[9000]绑定成功!

10月 20, 2024 6:15:21 下午 io.netty.handler.logging.LoggingHandler channelRead

信息: [id: 0x283ea389, L:/[0:0:0:0:0:0:0:0]:9000] READ: [id: 0x56e80062, L:/127.0.0.1:9000 - R:/127.0.0.1:4682]

10月 20, 2024 6:15:21 下午 io.netty.handler.logging.LoggingHandler channelReadComplete

信息: [id: 0x283ea389, L:/[0:0:0:0:0:0:0:0]:9000] READ COMPLETE

PacketDecoder接收到的远方发来的byteBuf

+-----+

| 0 1 2 3 4 5 6 7 8 9 a b c d e f |

+-----+

|00000000| ab cd ab cd 01 01 01 00 00 00 4e 7b 22 63 6f 6d |.....N{"com|

|00000010| 6d 61 6e 64 22 3a 31 2c 22 70 61 73 73 77 6f 72 |mand":1,"passwor|

|00000020| 64 22 3a 22 31 32 33 34 35 36 22 2c 22 75 73 65 |d":"123456","use|

|00000030| 72 49 64 22 3a 31 30 30 30 2c 22 75 73 65 72 4e |rId":1000,"userN|

|00000040| 61 6d 65 22 3a 22 61 64 6d 69 6e 22 2c 22 76 65 |ame":"admin","ve|

|00000050| 72 73 69 6f 6e 22 3a 31 7d |rsion":1} |

+-----+

PacketDecoder反序列化得到的Packet:LoginRequestPacket(userId=1000, userName=admin, password=123456)

LoginRequestHandler收到:LoginRequestPacket(userId=1000, userName=admin, password=123456)

LoginRequestHandler生成响应对象:LoginResponsePacket(isSuccess=false, message>LoginRequestHandler:登录失败, 请检查用户名与密码是否正确。)

PacketEncoder收到的Packet:LoginResponsePacket(isSuccess=false, message>LoginRequestHandler:登录失败, 请检查用户名与密码是否正确。)

成功登录后的客户端

```
Run  MyNettyServer x  MyNettyClient x
[C:\Program Files\Java\jdk-21\bin\java.exe" ...
PacketEncoder收到的Packet:LoginRequestPacket(userId=1000, userName=testUser, password=123456)
连接成功
PacketDecoder接收到的远方发来的byteBuf
+-----+
| 0 1 2 3 4 5 6 7 8 9 a b c d e f |
+-----+
|00000000| ab cd ab cd 01 01 02 00 00 00 5a 7b 22 63 6f 6d |.....Z{"com|
|00000010| 6d 61 6e 64 22 3a 32 2c 22 69 73 53 75 63 63 65 |mand":2,"isSucce|
|00000020| 73 73 22 3a 74 72 75 65 2c 22 6d 65 73 73 61 67 |ss":true,"messag|
|00000030| 65 22 3a 22 4c 6f 67 69 6e 52 65 71 75 65 73 74 |e":"LoginRequest|
|00000040| 48 61 6e 64 6c 65 72 3a e7 99 bb e5 bd 95 e6 88 |Handler:.....|
|00000050| 90 e5 8a 9f ef bc 81 22 2c 22 76 65 72 73 69 6f |.....","versio|
|00000060| 6e 22 3a 31 7d |n":1} |
+-----+
PacketDecoder反序列化得到的Packet:LoginResponsePacket(isSuccess=true, message=LoginRequestHandler:登录成功! )
LoginResponseHandler收到数据: LoginResponsePacket(isSuccess=true, message=LoginRequestHandler:登录成功! )
LoginResponseHandler设置登录结果: success
```

小结



本讲通过一个比较复杂的示例，展示了如何在客户端与服务端之间使用自定义通讯协议进行信息交换的基本方法，这个实例非常典型，具有相当的灵活性与可扩展性，可以在本讲示例的基础之上，开发更复杂的网络应用程序。



关于本讲示例，其实还留有一个可扩展点，那就是将检查用户是否登录的功能独立抽取为一个Handler加入到管线中，当用户登录成功之后，可以调用 `pipeline().remove()` 方法移除这个Handler，因为既然已经登录成功了，就没有必要为后继的每个请求都去检查是否登录。